

從 App Engine Memcache 遷移至 Cloud Memorystore (模組 13)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-13-memorystore?hl=zh-tw>

步驟數：8

瞭解如何遷移 Python 2 App Engine NDB & 將 Memcache 應用程式遷移至 Cloud NDB (&A)；Cloud Memorystore (for Redis)，再升級至 Python 3

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 設定快取服務](#)
5. [5. 更新設定檔](#)
6. [6. 更新應用程式檔案](#)
7. [7. 摘要/清除](#)
8. [8. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室旨在向 Python 2 App Engine 開發人員說明如何從 App Engine Memcache 遷移至 [Cloud Memorystore](#) (適用於 [Redis](#))。此外，從 App Engine `ndb` 到 [Cloud NDB](#) 也有[隱含遷移](#)，但這主要涵蓋在[第 2 堂程式碼實驗室](#)中；如需更多逐步資訊，請參閱該實驗室。

未使用 Memcache 嗎？

如果您的應用程式未使用 App Engine Memcache，可以略過這個程式碼研究室，或將其當做練習，熟悉如何[遷移至 Cloud Memorystore](#)。

在接下來的研究室中

- 設定 Cloud Memorystore 執行個體 (透過 Cloud 控制台或 `gcloud` 工具)
- 設定 Cloud 無伺服器 VPC 存取連接器 (透過 Cloud Console 或 `gcloud` 工具)
- 從 App Engine Memcache 遷移至 Cloud Memorystore
- 在範例應用程式中，使用 Cloud Memorystore 實作快取功能
- 從 App Engine `ndb` 遷移至 Cloud NDB

軟硬體需求

- 已啟用[有效帳單帳戶](#)的 [Google Cloud 專案](#) (這不是免費的程式碼研究室)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發及部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 12 堂課 App Engine 應用程式 (完成[第 12 堂課程式碼研究室](#) [建議]，或從存放區複製[第 12 堂課應用程式](#))

問卷調查

2. 背景

本程式碼研究室將示範如何將範例應用程式從 App Engine Memcache (和 NDB) 遷移至 Cloud Memorystore (和 Cloud NDB)。這個程序會取代 App Engine 套裝組合服務的依附元件，讓應用程式更具可攜性。您可以選擇繼續使用 App Engine，也可以考慮改用上述任一替代方案。

與本系列的其他遷移作業相比，這項作業需要投入更多心力。建議您改用 [Cloud Memorystore](#)，這是全代管的雲端快取服務。Memorystore 支援一對熱門的開放原始碼快取引擎：[Redis](#) 和 [Memcached](#)。這個遷移模組使用 [Cloud Memorystore for Redis](#)。詳情請參閱 [Memorystore 和 Redis 總覽](#)。

由於 Memorystore 需要執行中的伺服器，因此也需要 [Cloud VPC](#)。具體來說，您必須建立 [Serverless VPC Access 連接器](#)，App Engine 應用程式才能透過 Memorystore 執行個體的私人 IP 位址連線。完成這項練習後，您將更新應用程式，使其行為與先前相同，但會以 Cloud Memorystore 做為快取服務，取代 App Engine 的 Memcache 服務。

本教學課程會先使用 [Python 2 的第 12 堂課範例應用程式](#)，然後再進行 Python 3 的選用次要升級。如果您已熟悉如何透過 [Python 3 App Engine SDK](#)，從 [Python 3 存取 App Engine 綁定的服務](#)，可以改為從 [第 12 堂課範例應用程式的 Python 3 版本](#) 開始。這麼做會導致您無法使用 SDK，因為 Memorystore 「並非」App Engine 隨附服務。本教學課程不會說明如何使用 Python 3 App Engine SDK。

本教學課程包含下列主要步驟：

1. 設定/準備工作
 2. 設定快取服務
 3. 更新設定檔
 4. 更新主要應用程式
-

3. 設定/準備工作

準備 Cloud 專案

建議您重複使用完成第 12 堂程式碼研究室時使用的專案。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。本系列程式碼研究室的每個單元都有「START」（開始使用的基準程式碼）和「FINISH」（遷移後的應用程式）。我們提供 FINISH 程式碼，方便您在遇到問題時，比較自己的解決方案和我們的解決方案。如有任何問題，隨時可以回溯至「開始」狀態。這些檢查點旨在確保您順利學會如何執行遷移作業。

無論使用哪個 Cloud 專案，請務必[啟用帳單帳戶](#)。此外，請務必[啟用 App Engine](#)。請詳閱並瞭解執行這些教學課程的[一般費用影響](#)。不過，與本系列的其他程式碼研究室不同，本程式碼研究室使用的 Cloud 資源沒有免費層級，因此完成練習會產生一些費用。我們會提供更具體的費用資訊，以及減少用量的建議，包括在結尾提供釋出資源的說明，盡量減少帳單費用。

取得基準範例應用程式

本程式碼研究室會從第 12 模組的基準程式碼開始，逐步說明遷移程序。完成後，您會得到可運作的第 13 課應用程式，與其中一個 FINISH 資料夾中的程式碼非常相似。這些資源包括：

- 開始：[Python 2 第 12 個模組](#) (mod12) 或 [Python 3](#) (mod12b) 應用程式
- 完成：[模組 13 Python 2](#) (mod13a) 或 [Python 3](#) (mod13b) 應用程式
- [整個遷移存放區](#) (複製或下載 ZIP 檔案)

START 資料夾應包含下列檔案：

```
$ ls
README.md          app.yaml           main.py
requirements.txt   templates
```

如果您是從 Python 2 版本開始，也會有 `appengine_config.py` 檔案，而且如果您已完成第 12 個程式碼研究室，可能會有 `lib` 資料夾。

(重新) 部署第 12 個模組的應用程式

剩餘的準備步驟：

- 重新熟悉 `gcloud` 指令列工具 (如有需要)
- (視需要) 將第 12 模組的程式碼重新部署至 App Engine

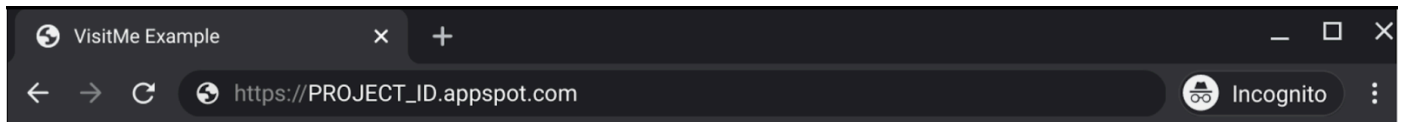
Python 2 使用者應使用下列指令刪除並重新安裝 `lib` 資料夾：

```
rm -rf ./lib; pip install -t lib -r requirements.txt
```

現在所有使用者 (Python 2 和 3 使用者) 都應使用下列指令將程式碼上傳至 App Engine：

```
gcloud app deploy
```

成功部署後，請確認應用程式的外觀和功能與第 12 模組中的應用程式相同，也就是追蹤造訪次數的網頁應用程式，並為同一位使用者快取一小時：



VisitMe example

Last 10 visits

- Tue Mar 30 06:49:20 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36
- Sun Mar 28 03:45:17 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
- Sat Mar 27 20:31:55 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.0
- Sat Mar 27 20:23:39 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
- Fri Mar 26 04:09:23 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Fri Mar 26 04:07:26 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Sat Mar 20 00:17:54 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Sat Mar 20 00:02:06 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
- Sat Mar 20 00:00:15 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
- Fri Mar 19 23:28:54 2021 from 127.0.0.1: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36

由於系統會快取最近的造訪記錄，因此重新整理頁面時，載入速度應該相當快。



4. 設定快取服務

Cloud Memorystore「並非」無伺服器服務。您必須提供執行個體，在本例中為執行中的 Redis。與 Memcache 不同，Memorystore 是獨立的 Cloud 產品，**沒有免費層級**，因此請務必先查看 [Memorystore for Redis 定價資訊](#)，再繼續操作。為盡量減少這項練習的成本，建議您使用最少的資源來運作：**基本服務層級**和 **1 GB** 容量。

Memorystore 執行個體與 App Engine 應用程式 (執行個體) 位於不同網路，因此必須建立 [Serverless VPC Access 連接器](#)，App Engine 才能存取 Memorystore 資源。為盡量降低 VPC 成本，請選擇執行個體類型 (**f1-micro**)，並要求最少數量的執行個體 (建議最少 2 個，最多 3 個)。另請參閱 [VPC 定價資訊頁面](#)。

費用：本教學課程並非免費

本系列的大部分遷移程式碼研究室都可免費完成，但本程式碼研究室是例外。Cloud Memorystore 和 Cloud VPC 均未提供**免費方案**，因此您必須支付部分費用才能繼續操作。強烈建議您先查看這兩項產品的價格資訊，再繼續操作。如果對費用很敏感，但仍想繼續使用這項功能，建議採用費用最低的選項。為盡量減少費用，請在完成程式碼研究室後立即釋出這些資源。

在引導您建立各項必要資源時，我們會重複提供這些降低費用的建議。此外，在 Cloud 控制台建立 Memorystore 和 VPC 資源時，右上角會顯示各項產品的定價計算機，提供每月費用預估值 (請見下圖)。變更選項時，這些值會自動調整。您應該會看到類似下列內容：

Summary

Tier	Basic
Location	us-central1
Estimated maximum throughput (MB/s) ⓘ	500 / 2000

Cost estimate

Based on instance tier, region, and capacity. [Pricing details](#)

1 GB  \$35.77/month

Cloud Memorystore
pricing calculator

Details

Estimated charges  USD \$11.10 - \$16.64/month
Plus network costs

Estimated bandwidth

100 Mbps at minimum instances (\$11.10)

150 Mbps at maximum instances (\$16.64)

Cloud VPC
pricing calculator

這兩項資源都必須建立，先建立哪一個都沒關係。如果您先建立 Memorystore 執行個體，App Engine 應用程式就無法在沒有 VPC 連接器的情況下連線至該執行個體。同樣地，如果您先建立虛擬私有雲連接器，App Engine 應用程式也無法與該虛擬私有雲網路通訊。本教學課程會先建立 Memorystore 執行個體，再建立 VPC 連接器。

兩項資源都上線後，您要將相關資訊新增至 `app.yaml`，應用程式才能存取快取。您也可以參閱官方文件中的 [Python 2](#) 或 [Python 3](#) 指南。此外，也建議參閱 Cloud NDB 遷移頁面上的資料快取指南 ([Python 2](#) 或 [Python 3](#))。

在 App Engine 區域中建立所有資源

您將在下方建立所有額外的 Cloud 資源，以便將 Cloud Memorystore 做為 App Engine 應用程式的快取服務。請注意，您必須在與 App Engine 應用程式相同的區域中，建立 Memorystore 執行個體和無伺服器虛擬私有雲存取連接器。

建立 Cloud Memorystore 執行個體

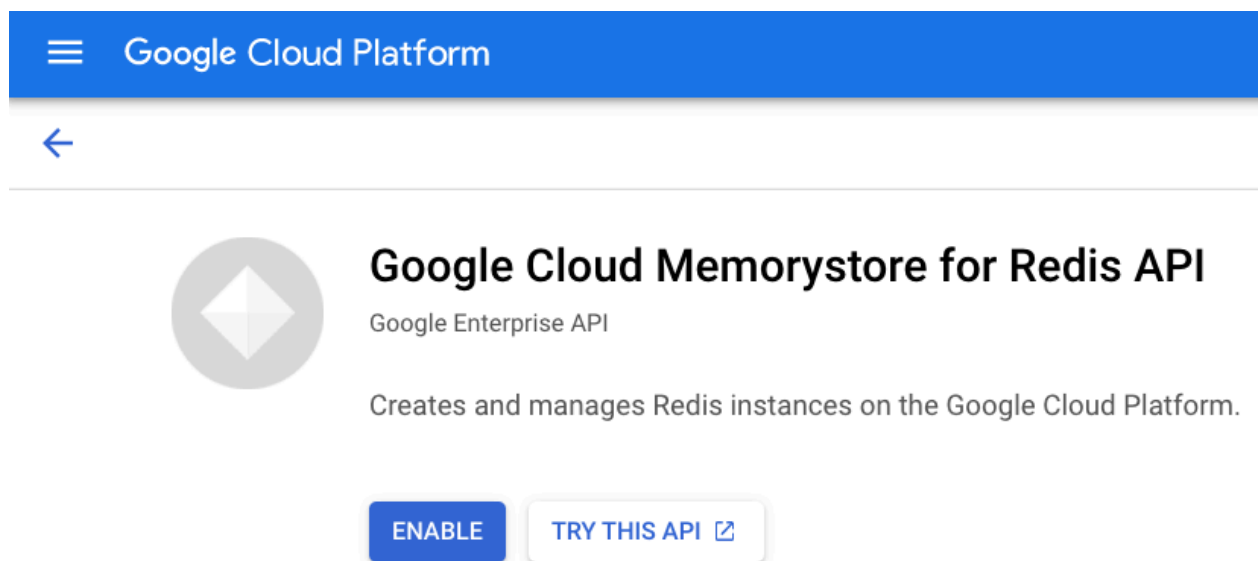
由於 Cloud Memorystore 沒有[免費方案](#)，建議您分配最少量的資源來完成 Codelab。您可以透過下列設定，盡量降低費用：

- 選取最低服務層級：**基本** (控制台預設為「標準」，`gcloud` 預設為「基本」)。
- 選擇最少的儲存空間：**1 GB** (主機預設為 16 GB，`gcloud` 預設為 1 GB)。
- 通常任何軟體的最新版本都需要最多資源，但我們也不建議選取最舊版本。目前第二個最新版本為 **Redis 版本 5.0** (管理中心預設版本：6.x)

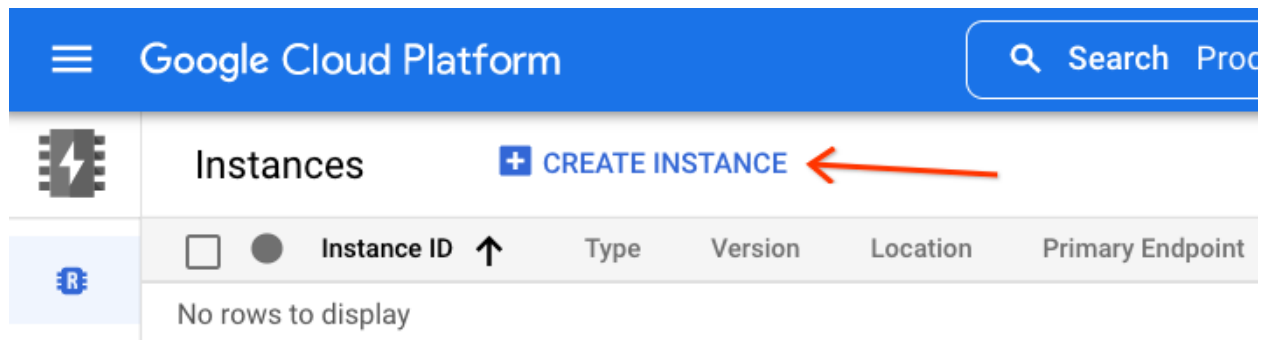
請記住這些設定，下一節將引導您透過 Cloud Console 建立執行個體。如果您偏好透過指令列執行這項操作，請跳到下一個步驟。

透過 Cloud 控制台

前往 Cloud 控制台的 [Cloud Memorystore 頁面](#) (系統可能會提示您提供帳單資訊)。如果尚未啟用 Memorystore，系統會提示您啟用：



啟用後 (可能需要一併啟用帳單)，您會來到 Memorystore 資訊主頁。您可以在這裡查看專案中建立的所有執行個體。下方顯示的專案沒有任何資料，因此您會看到「沒有要顯示的資料列」。如要建立 Memorystore 執行個體，請按一下頂端的「建立執行個體」：



Cloud 控制台更新

請注意，Cloud 控制台的使用者介面會不時變更，因此可能與這裡的螢幕截圖不完全相同。

這個頁面會顯示表單，請填入所需設定，建立 Memorystore 執行個體：

Cloud Memorystore for Redis is a fully managed Redis service for the Google Cloud Platform. [Learn more](#)

Name your instance

The instance ID is a permanent and unique identifier. The display name is optional and for display purposes only.

Instance ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Display name

Set instance properties

Tier *
Determines availability, cost, and performance.

☐ Basic
Lower cost. Does not provide high availability.

☒ Standard
Supports automatic failover for high availability and up to 5 read replicas for scaling reads. [Learn more.](#)

為降低本教學課程和範例應用程式的費用，請按照先前介紹的建議操作。選好之後，按一下「建立」。建立程序需要幾分鐘才能完成。完成後，複製執行個體的 IP 位址和通訊埠號碼，然後新增至 `app.yaml`。

使用指令列

透過 Cloud 控制台建立 Memorystore 執行個體，可提供視覺化資訊，但有些人偏好使用指令列。請務必先 [安裝](#) 及 [初始化](#) gcloud，再繼續操作。

與 Cloud 控制台一樣，您必須啟用 Cloud Memorystore for Redis。發出 `gcloud services enable redis.googleapis.com` 指令並等待完成，如下列範例所示：

```
$ gcloud services enable redis.googleapis.com
Operation "operations/acat.p2-aaa-bbb-ccc-ddd-eee-ffffff" finished successfully.
```

如果服務已啟用，(再次) 執行指令不會產生 (負面) 副作用。啟用服務後，我們來建立 Memorystore 執行個體。該指令如下所示：

```
gcloud redis instances create NAME --redis-version VERSION \
  --region REGION --project PROJECT_ID
```

為您的 Memorystore 執行個體選擇名稱；本實驗室使用「demo-ms」做為名稱，專案 ID 為「my-project」。這個範例應用程式的區域是 `us-central1` (與 `us-central` 相同)，但如果擔心延遲問題，可以使用離您較近的區域。您必須選取與 App Engine 應用程式相同的地區。您可以選取偏好的任何 Redis 版本，但我們將使用先前建議的 5 版。根據這些設定，您會發出下列指令 (以及相關聯的輸出內容)：

```
$ gcloud redis instances create demo-ms --region us-central1 \
  --redis-version redis_5_0 --project my-project

Create request issued for: [demo-ms]
Waiting for operation [projects/my-project/locations/us-central1/operations/operation-xxxx] to complete...done.
Created instance [demo-ms].
```

與 Cloud 控制台預設值不同，gcloud 預設為最少資源。因此，該指令不需要服務層級或儲存空間量。建立 Memorystore 執行個體需要幾分鐘，完成後請記下執行個體的 IP 位址和連接埠號碼，稍後會將這些資訊新增至 `app.yaml`。

確認建立執行個體

透過 Cloud 控制台或指令列

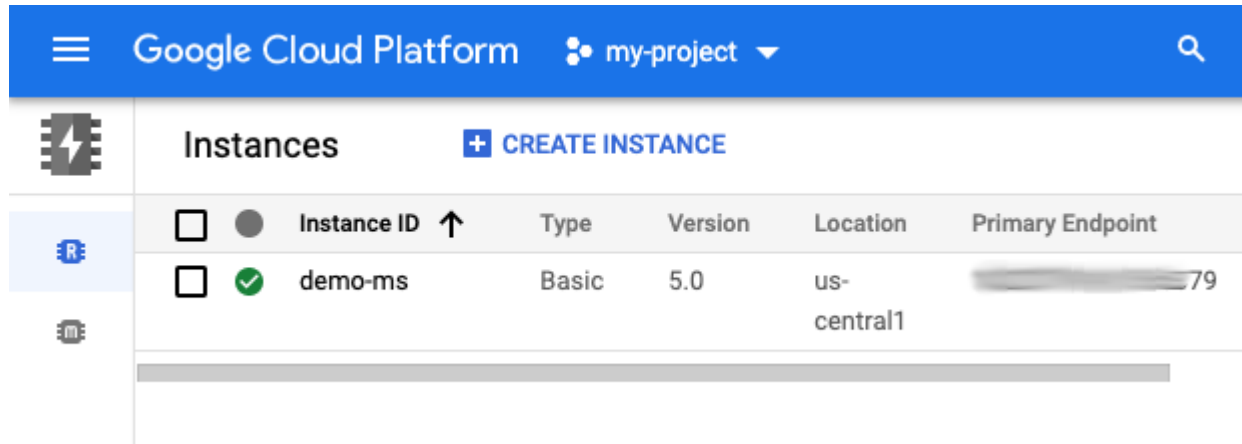
無論您是透過 Cloud Console 或指令列建立執行個體，都可以使用以下指令確認執行個體是否可用：`gcloud redis instances list --region REGION`

以下指令可檢查 `us-central1` 地區中的執行個體，以及顯示我們剛建立執行個體的預期輸出內容：

```
$ gcloud redis instances list --region us-central1
```

INSTANCE_NAME	VERSION	REGION	TIER	SIZE_GB	HOST	PORT	NETWORK
RESERVED_IP	STATUS	CREATE_TIME					
demo-ms	REDIS_5_0	us-central1	BASIC	1	10.aa.bb.cc	6379	default
10.aa.bb.dd/29	READY	2022-01-28T09:24:45					

系統要求提供執行個體資訊或設定應用程式時，請務必使用 `HOST` 和 `PORT` (而非 `RESERVED_IP`)。Cloud 控制台中的 Cloud Memorystore 資訊主頁現在應會顯示該執行個體：



Instance ID	Type	Version	Location	Primary Endpoint
demo-ms	Basic	5.0	us-central1	79

從 Compute Engine 虛擬機器

如果您有 Compute Engine 虛擬機器 (VM)，也可以從 [VM 傳送 Memorystore 執行個體直接指令](#)，確認執行個體是否正常運作。請注意，使用 VM 可能會產生相關費用，與您已使用的資源無關。

建立無伺服器 VPC 存取連接器

與 Cloud Memorystore 相同，您可以在 Cloud Console 或指令列中建立無伺服器 Cloud VPC 連接器。同樣地，Cloud VPC 沒有[免費層級](#)，因此建議您分配最少量的資源來完成 Codelab，以盡量降低成本。您可以透過下列設定達成這個目標：

- 選取最低的執行個體數量上限：**3** (控制台和 `gcloud` 預設值：10)
- 選擇最低成本的機器類型：**f1-micro** (控制台預設值：`e2-micro`，沒有 `gcloud` 預設值)

下一節將引導您使用上述 Cloud 虛擬私有雲設定，從 Cloud 控制台建立連接器。如要透過指令列執行這項操作，請跳至下一節。

透過 Cloud 控制台

前往 Cloud 控制台的 [Cloud Networking「無伺服器 VPC 存取」頁面](#) (系統可能會提示您提供帳單資訊)。如果尚未啟用 API，系統會提示您啟用：



Serverless VPC Access API

Google Enterprise API

API for managing VPC access connectors.

ENABLE

啟用 API (可能需要一併啟用帳單) 後，您會看到顯示所有已建立 VPC 連接器的資訊主頁。下方螢幕截圖中使用的專案沒有任何資料列，因此才會顯示「沒有要顯示的資料列」。在控制台中，按一下頂端的「建立連結器」：

Google Cloud Platform

Search Products

Serverless VPC access

+ CREATE CONNECTOR

DELETE

Serverless VPC Access allows Cloud Functions, Cloud Run (fully managed) services and App Engine standard environment apps to access resources in a VPC network using those resources' private IPs. [Learn more](#)

Filter

Enter property name or value

☐	☒ Name ↑	Subnet	Region	Instance type
No rows to display				

填寫表單，設定所需設定：

Google Cloud Platform

Search Products,

← Create connector

Name *

Region *
us-central1

A region is a specific geographical location where you can run your resources.

Network *
default

Subnet *
Select an unused /28 subnet or create a new one by entering an unused /28 IP range. The VPC Connector will create connector instances on this subnet.

SHOW SCALING SETTINGS

CREATE

CANCEL

為自己的應用程式選擇合適的設定。在本教學課程中，範例應用程式的需求極少，因此應盡量降低成本，請按照先前的建議操作。選取完畢後，按一下「建立」。申請虛擬私有雲連接器需要幾分鐘才能完成。

從指令列

建立 VPC 連接器前，請先啟用無伺服器 VPC 存取 API。發出下列指令後，您應該會看到類似的輸出內容：

```
$ gcloud services enable vpcaccess.googleapis.com
Operation "operations/acf.p2-aaa-bbb-ccc-ddd-eee-ffffff" finished successfully.
```

啟用 API 後，請使用下列指令建立 VPC 連接器：

```
gcloud compute networks vpc-access connectors create CONNECTOR_NAME \
  --range 10.8.0.0/28 --region REGION --project PROJECT_ID
```

為連接器選擇名稱，以及未使用的 /28 CIDR 區塊起始 IP 位址。本教學課程假設：

- 專案 ID：my-project
- VPC 連接器名稱：demo-vpc
- 執行個體數量下限：2 個 (預設值)，執行個體數量上限：3 個
- 執行個體類型：f1-micro
- Region (區域)：us-central1

- **IPv4 CIDR 區塊**：10.8.0.0/28 (如雲端控制台建議)

如果您執行下列指令時，已考量上述假設，輸出內容應如下所示：

```
$ gcloud compute networks vpc-access connectors create demo-vpc \
  --max-instances 3 --range 10.8.0.0/28 --machine-type f1-micro \
  --region us-central1 --project my-project

Create request issued for: [demo-vpc]
Waiting for operation [projects/my-project/locations/us-central1/operations/xxx] to
complete...done.
Created connector [demo-vpc].
```

上述指令省略了指定預設值，例如 2 個執行個體的最小值，以及名為 `default` 的網路。建立 VPC 連接器需要幾分鐘的時間。

確認已建立連接器

程序完成後，請發出下列 `gcloud` 指令 (假設是 `us-central1` 區域)，確認已建立並可供使用：

```
$ gcloud compute networks vpc-access connectors list --region us-central1
CONNECTOR_ID  REGION      NETWORK  IP_CIDR_RANGE  SUBNET  SUBNET_PROJECT
MIN_THROUGHPUT  MAX_THROUGHPUT  STATE
demo-vpc      us-central1  default  10.8.0.0/28    200
300           READY
```

同樣地，資訊主頁現在應該會顯示您剛建立的連結器：

Name	Network	Subnet	IP address range	Region	Instance type
demo-vpc	default		10.8.0.0/28	us-central1	f1-micro

記下 Cloud 專案 ID、虛擬私有雲連接器名稱和區域。

您已透過指令列或控制台建立必要的額外 Cloud 資源，現在可以更新應用程式設定，支援使用這些資源。

5. 更新設定檔

首先，請對設定檔進行所有必要的更新。本程式碼研究室的主要目標是協助 Python 2 使用者遷移，但通常在下方各節中，也會提供進一步移植到 Python 3 的相關資訊。

requirements.txt

在本節中，我們將新增支援 Cloud Memorystore 和 Cloud NDB 的套件。如果是 Cloud Memorystore *for Redis*，只要使用 Python 的標準 Redis 用戶端 (`redis`) 即可，因為沒有專屬的 Cloud Memorystore 用戶端程式庫。將 `redis` 和 `google-cloud-ndb` 附加至 `requirements.txt`，並從模組 12 加入 `flask`：

```
flask
redis
google-cloud-ndb
```

這個 `requirements.txt` 檔案沒有任何版本號碼，表示系統會選取最新版本。如有任何不相容問題，請指定版本號碼，鎖定可正常運作的版本。

app.yaml

要新增的章節

使用 Cloud NDB 等 Cloud API 時，Python 2 App Engine 執行階段需要特定第三方套件，即 `grpcio` 和 `setuptools`。Python 2 使用者必須在 `app.yaml` 中列出這類內建程式庫，以及[可用版本](#)。如果還沒有 `libraries` 區段，請建立一個，並新增下列兩個程式庫：

```
libraries:
- name: grpcio
  version: latest
- name: setuptools
  version: latest
```

遷移應用程式時，應用程式可能已有 `libraries` 區段。如果有的話，且缺少 `grpcio` 和 `setuptools`，請將這兩項新增至現有的 `libraries` 區段。

接下來，範例應用程式需要 Cloud Memorystore 執行個體和 VPC 連接器資訊，因此請在 `app.yaml` 中新增下列兩個部分，無論您使用哪個 Python 執行階段都一樣：

```
env_variables:
  REDIS_HOST: 'YOUR_REDIS_HOST'
  REDIS_PORT: 'YOUR_REDIS_PORT'

vpc_access_connector:
  name: projects/PROJECT_ID/locations/REGION/connectors/CONNECTOR
```

必要更新到此完成。更新後的 `app.yaml` 應如下所示：

```

runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

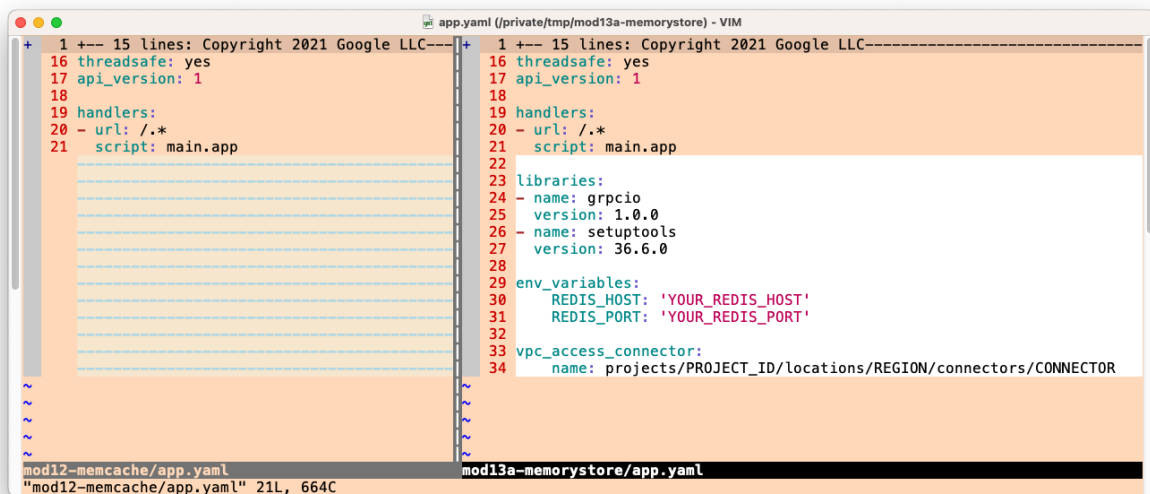
libraries:
- name: grpcio
  version: 1.0.0
- name: setuptools
  version: 36.6.0

env_variables:
  REDIS_HOST: 'YOUR_REDIS_HOST'
  REDIS_PORT: 'YOUR_REDIS_PORT'

vpc_access_connector:
  name: projects/PROJECT_ID/locations/REGION/connectors/CONNECTOR

```

以下是「前後對照」的範例，說明您應套用至 `app.yaml` 的更新：



*Python 3 差異

本節為選用內容，僅適用於要移植到 Python 3 的情況。如要這麼做，請對 Python 2 設定進行多項變更。如果目前不打算升級，請略過這個部分。

Python 3 執行階段不會使用 `threadsafe` 和 `api_version`，因此請刪除這兩項設定。最新版 App Engine 執行階段不支援內建第三方程式庫，也不支援複製非內建程式庫。第三方套件的唯一規定是將其列於 `requirements.txt` 中。因此可以刪除 `app.yaml` 的整個 `libraries` 區段。

接著，Python 3 執行階段需要使用自行進行路由的網頁架構，因此我們在第 1 模組中向開發人員說明如何從 `webp2` 遷移至 `Flask`。因此，所有指令碼處理常式都必須變更為 `auto`。由於這個應用程式不會提供任何靜態檔案，因此列出處理常式「毫無意義」（因為全都是 `auto`），所以整個 `handlers` 區段也可以移除。因此，為 Python 3 調整的新縮寫 `app.yaml` 應縮短為如下所示：

```
runtime: python39

env_variables:
  REDIS_HOST: 'YOUR_REDIS_HOST'
  REDIS_PORT: 'YOUR_REDIS_PORT'

vpc_access_connector:
  name: projects/PROJECT_ID/locations/REGION/connectors/CONNECTOR
```

將程式碼移植到 Python 3 時，`app.yaml` 的差異摘要如下：

- 刪除 `threadsafe` 和 `api_version` 設定
- 刪除「`libraries`」部分
- 刪除 `handlers` 區段 (或如果應用程式提供靜態檔案，則只刪除 `script` 處理常式)

替換值

Memorystore 和 VPC 連接器新區段中的值只是預留位置。將這些大寫值 (`YOUR_REDIS_HOST`, `YOUR_REDIS_PORT`, `PROJECT_ID`, `REGION`, `CONNECTOR_NAME`) 替換為您先前建立這些資源時儲存的值。就 Memorystore 執行個體而言，請務必使用 `HOST` (而非 `RESERVED_IP`) 和 `PORT`。以下是透過指令列快速取得 `HOST` 和 `PORT` 的方法，假設執行個體名稱為 `demo-ms`，且 `REGION` 為 `us-central1`：

```
$ gcloud redis instances describe demo-ms --region us-central1 \
  --format "value(host,port)"
10.251.161.51    6379
```

如果範例 Redis 執行個體的 IP 位址為 `10.10.10.10`，使用專案 `my-project` 中位於 `us-central1` 區域的連接埠 `6379`，且 VPC 連接器名稱為 `demo-vpc`，則 `app.yaml` 中的這些部分會如下所示：

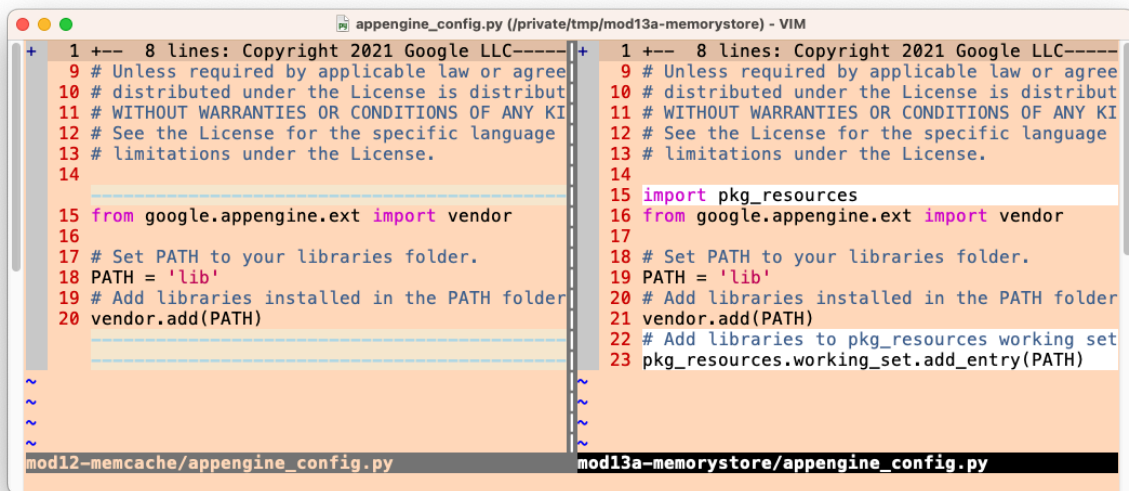
```
env_variables:
  REDIS_HOST: '10.10.10.10'
  REDIS_PORT: '6379'

vpc_access_connector:
  name: projects/my-project/locations/us-central1/connectors/demo-vpc
```

建立或更新 `appengine_config.py`

新增對內建第三方程式庫的支援

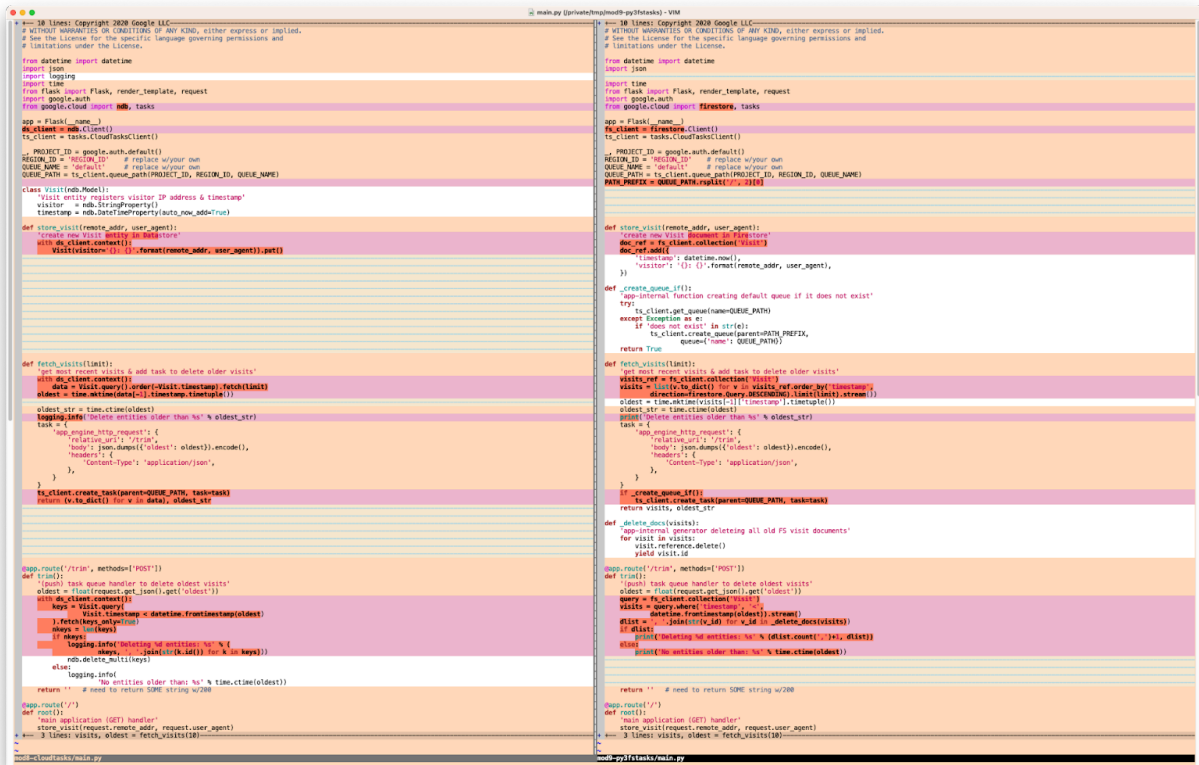
就像先前使用 `app.yaml` 時一樣，新增 `grpcio` 和 `setuptools` 程式庫的使用方式。修改 `appengine_config.py`，支援內建第三方程式庫。如果您覺得很熟悉，是因為在第 2 個單元中，從 App Engine `ndb` 遷移至 Cloud NDB 時，也需要執行這項操作。確切的變更內容是將 `lib` 資料夾新增至 `setuptools.pkg_resources` 工作集：



步驟 6 · 約 6 分鐘

6. 更新應用程式檔案

只有一個應用程式檔案 `main.py`，因此本節中的所有變更只會影響該檔案。我們提供圖片，說明將這個應用程式遷移至 Cloud Memorystore 時會進行的變更。僅供參考，不適合仔細分析。所有工作都在於我們對程式碼所做的變更。



讓我們從最上方開始，逐一處理這些部分。

更新匯入作業

第 12 堂課程的 `main.py` 匯入部分使用 Cloud NDB 和 Cloud Tasks，以下是匯入項目：

BEFORE:

```
from flask import Flask, render_template, request
from google.appengine.api import memcache
from google.appengine.ext import ndb
```

切換至 Memorystore 需要讀取環境變數，因此我們需要 Python `os` 模組和 Python Redis 用戶端 `redis`。由於 Redis 無法快取 Python 物件，請使用 `pickle` 封送處理最近造訪清單，因此也請匯入該清單。Memcache 的優點之一是物件序列化會自動進行，而 Memorystore 則比較偏向「DIY」。最後，請將 `ndb` 升級為 Cloud NDB，方法是將 `google.appengine.ext.ndb` 替換為 `google.cloud.ndb`。這些變更完成後，匯入內容應如下所示：

修改後：

```
import os
import pickle
from flask import Flask, render_template, request
from google.cloud import ndb
import redis
```

更新初始化作業

模組 12 初始化作業包括例項化 Flask 應用程式物件 `app`，以及設定一小時快取值的常數：

修改前：

```
app = Flask(__name__)
HOUR = 3600
```

使用 Cloud API 時需要用戶端，因此請在 Flask 之後立即例項化 Cloud NDB 用戶端。接著，從您在 `app.yaml` 中設定的環境變數，取得 Memorystore 執行個體的 IP 位址和通訊埠編號。取得這項資訊後，即可例項化 Redis 用戶端。更新後的程式碼如下所示：

修改後：

```
app = Flask(__name__)
ds_client = ndb.Client()
HOUR = 3600
REDIS_HOST = os.environ.get('REDIS_HOST', 'localhost')
REDIS_PORT = os.environ.get('REDIS_PORT', '6379')
REDIS = redis.Redis(host=REDIS_HOST, port=REDIS_PORT)
```

*遷移至 Python 3

這個部分是選用內容，適用於從 Python 3 版本的第 12 堂課應用程式開始學習的學員。如果是，您必須進行幾項與匯入和初始化相關的變更。

首先，由於 Memcache 是 App Engine 隨附服務，因此在 Python 3 應用程式中使用時，需要 App Engine SDK，具體來說，就是包裝 WSGI 應用程式 (以及其他[必要設定](#))：

BEFORE:

```
from flask import Flask, render_template, request
from google.appengine.api import memcache, wrap_wsgi_app
from google.appengine.ext import ndb

app = Flask(__name__)
app.wsgi_app = wrap_wsgi_app(app.wsgi_app)
HOUR = 3600
```

由於我們將遷移至 Cloud Memorystore (而非 Memcache 這類 App Engine 套裝組合服務)，因此必須移除 SDK 用法。這很簡單，只要刪除匯入 `memcache` 和 `wrap_wsgi_app` 的整行即可。同時刪除呼叫 `wrap_wsgi_app()` 的這一行。這些更新會讓應用程式的這部分 (實際上是整個應用程式) 與 Python 2 版本相同。

修改後：

```

import os
import pickle
from flask import Flask, render_template, request
from google.cloud import ndb
import redis

app = Flask(__name__)
ds_client = ndb.Client()
HOUR = 3600
REDIS_HOST = os.environ.get('REDIS_HOST', 'localhost')
REDIS_PORT = os.environ.get('REDIS_PORT', '6379')
REDIS = redis.Redis(host=REDIS_HOST, port=REDIS_PORT)

```

最後，請從 `app.yaml` 中移除 SDK 的使用情形 (刪除 `app_engine_apis: true` 這行)，並從 `requirements.txt` 中移除 (刪除 `appengine-python-standard` 這行)。

遷移至 Cloud Memorystore (和 Cloud NDB)

Cloud NDB 的資料模型與 App Engine `ndb` 的資料模型相容，因此 `Visit` 物件的定義保持不變。模仿第 2 模組遷移至 Cloud NDB 的做法，`store_visit()` 和 `fetch_visits()` 中的所有 Datastore 呼叫都會擴增並嵌入新的 `with` 區塊 (因為必須使用 Cloud NDB 內容管理員)。以下是這項變更前的呼叫：

BEFORE:

```

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    Visit(visitor='{}: {}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    return Visit.query().order(-Visit.timestamp).fetch(limit)

```

在兩個函式中新增 `with ds_client.context()` 區塊，並將 Datastore 呼叫放在裡面 (並縮排)。在這種情況下，呼叫本身不需要任何變更：

修改後：

```

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{}: {}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    with ds_client.context():
        return Visit.query().order(-Visit.timestamp).fetch(limit)

```

接下來，我們來看看快取變更。以下是第 12 堂課的 `main()` 函式：

BEFORE:

```
@app.route('/')
def root():
    'main application (GET) handler'
    # check for (hour-)cached visits
    ip_addr, usr_agt = request.remote_addr, request.user_agent
    visitor = '{}: {}'.format(ip_addr, usr_agt)
    visits = memcache.get('visits')

    # register visit & run DB query if cache empty or new visitor
    if not visits or visits[0].visitor != visitor:
        store_visit(ip_addr, usr_agt)
        visits = list(fetch_visits(10))
        memcache.set('visits', visits, HOUR) # set() not add()

    return render_template('index.html', visits=visits)
```

Redis 具有「get」和「set」呼叫，與 Memcache 相同。我們只需要替換對應的用戶端程式庫，對吧？就差一點，如先前所述，我們無法使用 Redis 快取 Python 清單（因為必須先序列化，而 Memcache 會自動處理這項作業），因此在 `set()` 呼叫中，請使用 `pickle.dumps()` 將造訪次數「封存」為字串。同樣地，從快取擷取造訪記錄時，您需要在 `get()` 之後，立即使用 `pickle.loads()` 取消封存。實作這些變更後，主要處理常式如下：

修改後：

```
@app.route('/')
def root():
    'main application (GET) handler'
    # check for (hour-)cached visits
    ip_addr, usr_agt = request.remote_addr, request.user_agent
    visitor = '{}: {}'.format(ip_addr, usr_agt)
    rsp = REDIS.get('visits')
    visits = pickle.loads(rsp) if rsp else None

    # register visit & run DB query if cache empty or new visitor
    if not visits or visits[0].visitor != visitor:
        store_visit(ip_addr, usr_agt)
        visits = list(fetch_visits(10))
        REDIS.set('visits', pickle.dumps(visits), ex=HOUR)

    return render_template('index.html', visits=visits)
```

至此，將範例應用程式的 Memcache 用法轉換為 Cloud Memorystore 的必要變更已完成。 `main.py` HTML 範本和移植到 Python 3 呢？

更新 HTML 範本檔案並移植到 Python 3？

驚喜大放送！由於應用程式的設計可在 Python 2 和 3 上執行，且不需要任何程式碼變更或相容性程式庫，因此這裡不需要執行任何動作。你會看到「`main.py`」。`mod13a` (2.x) 和 `mod13b` (3.x)「FINISH」資料夾中的檔案必須完全相同。`requirements.txt` 也適用相同規則，版本號碼如有差異則不在此限。由於使用者介面維持不變，`templates/index.html` 也不會更新。

先前在設定時，已完成在 Python 3 App Engine 上執行這個應用程式所需的一切作業：從 `app.yaml` 中移除不必要的指令，並刪除 `appengine_config.py` 和 `lib` 資料夾，因為這些在 Python 3 中不會用到。

App Engine NDB 和 Cloud NDB 提供自動快取功能

雖然我們明確要從 App Engine Memcache 遷移至 Cloud Memorystore，但請注意，App Engine `ndb` 和 Cloud NDB 都會分別為 Python 2 或 3 提供 Memcache 或 Memorystore 的自動快取功能。請參閱「[NDB 快取](#)」和「[啟用資料快取](#)」一文，瞭解 `ndb` 快取，後者的操作說明與本程式碼研究室的操作說明十分相似。如要瞭解其他功能，請參閱「[內容和交易選項](#)」。

我們沒有使用這項自動快取功能，是因為範例應用程式不會查詢個別 Datastore 實體 (這些實體可以透過快取輕鬆存取)。這個應用程式會快取實體陣列 (最近的造訪記錄)，但這類用途過於具體，自動快取層不支援。不過，您的應用程式可能會執行這類查詢，並運用這項功能，簡化整體使用體驗。

步驟 7 · 約 6 分鐘

7. 摘要/清除

本節將部署應用程式，並驗證應用程式是否正常運作，以及是否反映在輸出內容中，為本程式碼研究室畫下句點。應用程式驗證完成後，請執行任何清理作業，並考慮後續步驟。

部署及驗證應用程式

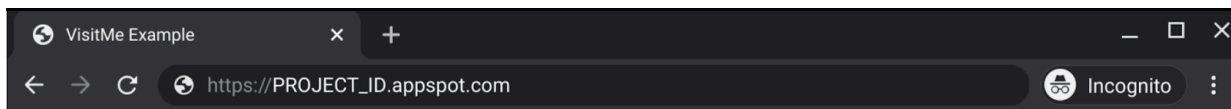
最後一個檢查步驟是部署範例應用程式。Python 2 開發人員：請使用下列指令刪除並重新安裝 `lib`。(如果系統同時安裝 Python 2 和 3，您可能需要明確執行 `pip2`)。

```
rm -rf ./lib
pip install -t lib -r requirements.txt
```

Python 2 和 3 開發人員現在都應使用下列指令部署應用程式：

```
gcloud app deploy
```

您只是在幕後重新連線，改用完全不同的快取服務，因此應用程式本身的運作方式應與第 12 堂課的應用程式相同：



VisitMe example

Last 10 visits

- Tue Mar 30 06:49:20 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36
- Sun Mar 28 03:45:17 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
- Sat Mar 27 20:31:55 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.0
- Sat Mar 27 20:23:39 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
- Fri Mar 26 04:09:23 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Fri Mar 26 04:07:26 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Sat Mar 20 00:17:54 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
- Sat Mar 20 00:02:06 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
- Sat Mar 20 00:00:15 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
- Fri Mar 19 23:28:54 2021 from 127.0.0.1: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36

這個步驟是程式碼研究室的最後一個步驟。建議您將更新後的範例應用程式與任一 Module 13 資料夾 ([mod13a](#) (Python 2) 或 [mod13b](#) (Python 3)) 進行比較。

清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生少量建構和儲存空間費用。Cloud Build 和 Cloud Storage 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `* LOC *`ation，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請關閉專案。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- Cloud Memorystore 需要執行個體，且沒有免費方案；如要進一步瞭解使用費用，請參閱定價頁面。
- Cloud Serverless VPC Access 連接器需要執行個體，且沒有免費方案。如要進一步瞭解使用費用，請參閱 Cloud VPC 定價頁面上的相關章節。
- Cloud Datastore (Cloud Firestore Datastore 模式) 提供免費方案；詳情請參閱定價頁面。

本教學課程使用了四項 Cloud 產品：

- App Engine
- Cloud Datastore
- Cloud Memorystore
- Cloud VPC

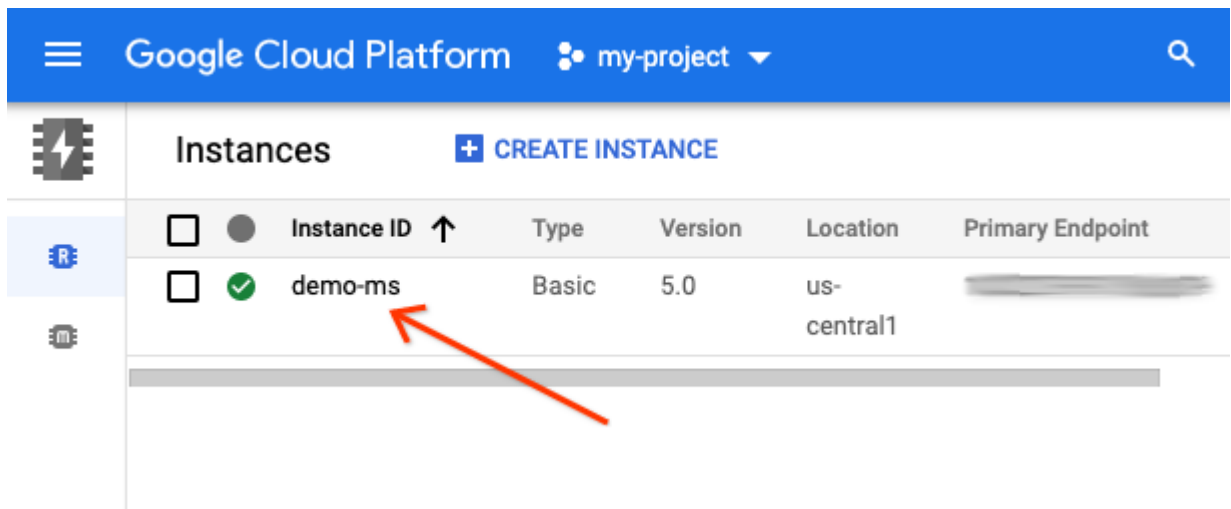
以下說明如何釋出這些資源，以免產生帳單費用或將費用降至最低。

關閉 Memorystore 執行個體和虛擬私有雲連接器

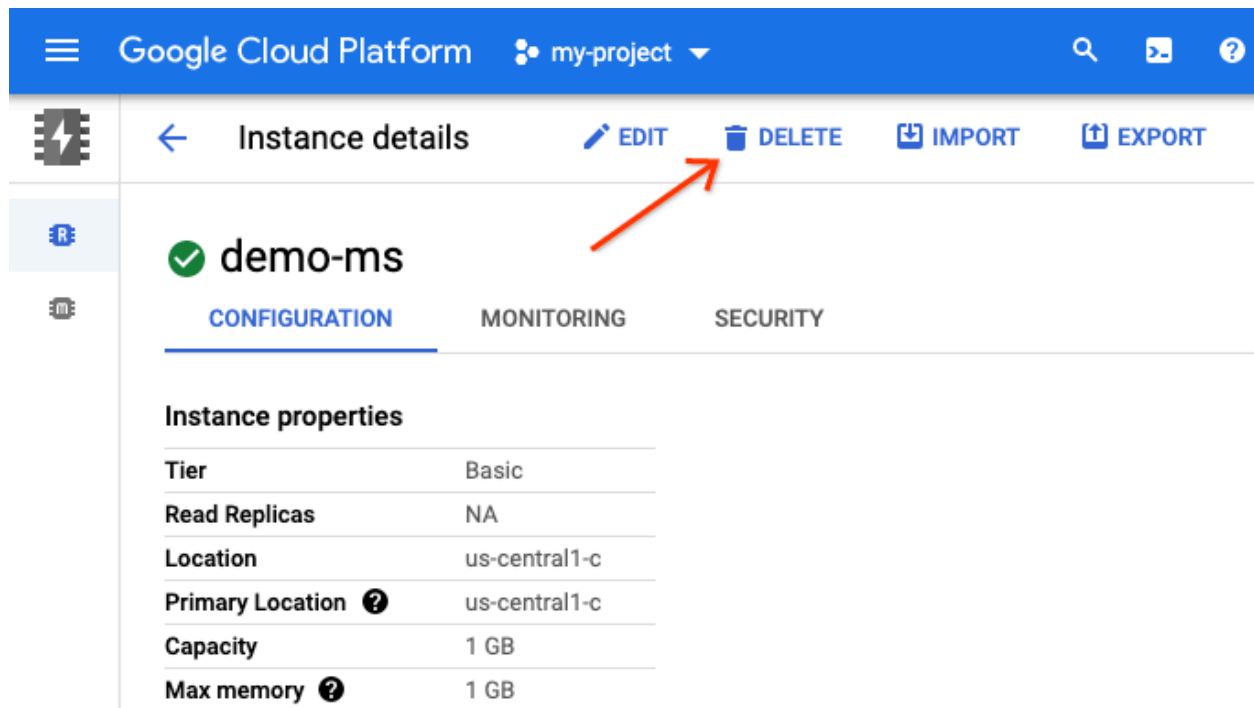
這些產品沒有免費方案，因此目前會產生費用。如果您未關閉 Cloud 專案 (請參閱下一節)，就必須刪除 Memorystore 執行個體和 VPC 連接器，才能停止計費。與建立這些資源時類似，您也可以從 Cloud Console 或指令列釋出這些資源。

透過 Cloud 控制台

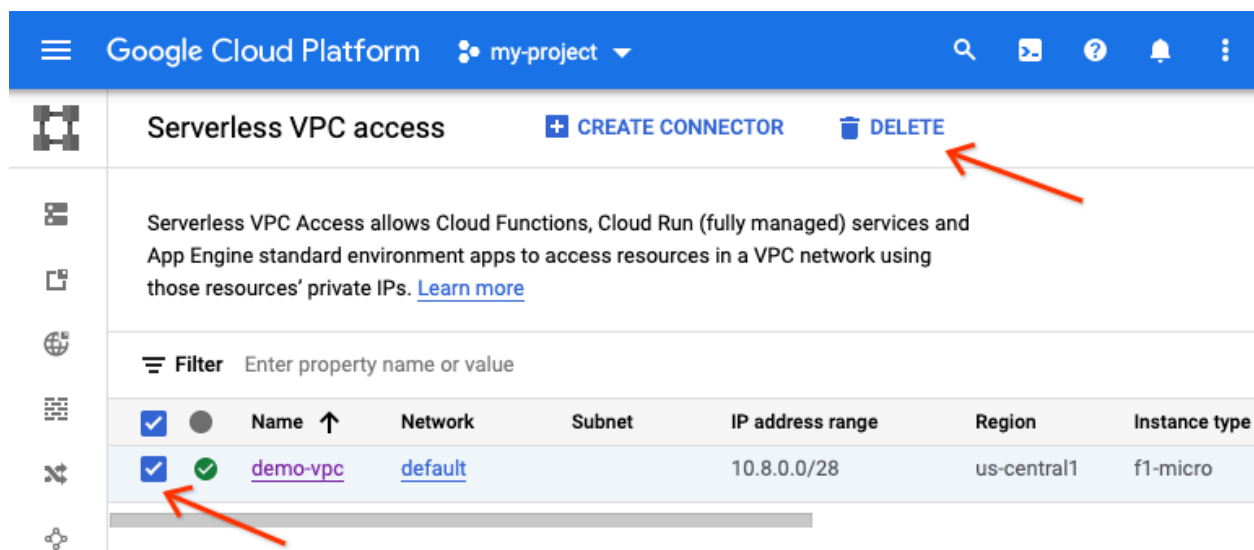
如要刪除 Memorystore 執行個體，請返回 Memorystore 資訊主頁，然後按一下執行個體 ID：



進入該執行個體的詳細資料頁面後，按一下「刪除」並確認：



如要刪除 VPC 連接器，請前往其資訊主頁，勾選要刪除的連接器旁邊的核取方塊，然後按一下「刪除」並確認：



透過指令列

下列 `gcloud` 指令會分別刪除 Memorystore 執行個體和 VPC 連接器：

- `gcloud redis instances delete INSTANCE --region REGION`
- `gcloud compute networks vpc-access connectors delete CONNECTOR --region REGION`

如果尚未透過 `gcloud config set project` 設定專案 ID，可能需要提供 `--project PROJECT_ID`。如果您的 Memorystore 執行個體名為 `demo-ms`，虛擬私有雲連接器名為 `demo-vpc`，且兩者都位於 `us-central1` 區域，請發出下列指令並確認：

```
$ gcloud redis instances delete demo-ms --region us-central1
You are about to delete instance [demo-ms] in [us-central1].
Any associated data will be lost.

Do you want to continue (Y/n)?

Delete request issued for: [demo-ms]
Waiting for operation [projects/PROJECT/locations/REGION/operations/operation-aaaaa-
bbbbbb-cccc--ddddd] to complete...done.
Deleted instance [demo-ms].
$
$ gcloud compute networks vpc-access connectors delete demo-vpc --region us-central1
You are about to delete connector [demo-vpc] in [us-central1].
Any associated data will be lost.

Do you want to continue (Y/n)?

Delete request issued for: [demo-vpc]
Waiting for operation [projects/PROJECT/locations/REGION/operations/aaaaa-bbbb-cccc-
dddd-eeee] to complete...done.
Deleted connector [demo-vpc].
```

每項要求都需要幾分鐘才能完成。如果您選擇關閉整個 Cloud 專案 (如前所述)，則可略過這些步驟，但系統仍會計費，直到關閉程序完成為止。

後續步驟

除了本教學課程外，您還可以參考其他遷移單元，瞭解如何從舊版套裝組合服務遷移：

- [單元 2](#)：從 App Engine `ndb` 遷移至 Cloud NDB
- [單元 7 至 9](#)：從 App Engine 工作佇列推送工作遷移至 Cloud Tasks
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 15 至 16](#)：從 App Engine Blobstore 遷移至 Cloud Storage
- [單元 18-19](#)：從 App Engine 工作佇列 (提取工作) 遷移至 Cloud Pub/Sub

App Engine 不再是 Google Cloud 中唯一的無伺服器平台。如果您有小型 App Engine 應用程式或功能有限的應用程式，並希望將其轉換為獨立的微服務，或是想將單體應用程式拆分成多個可重複使用的元件，這些都是考慮遷移至 [Cloud Functions](#) 的好理由。如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，建議遷移至 [Cloud Run](#)。下列單元會說明這些情況：

- 從 App Engine 遷移至 Cloud Functions：請參閱[第 11 堂課](#)
- 從 App Engine 遷移至 Cloud Run：請參閱[第 4 節](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 節](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile`s 的情況下完成這項作業。

您可以選擇改用其他無伺服器平台，但建議先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

8. 其他資源

以下列出其他資源，供開發人員進一步瞭解這個或相關的遷移模組，以及相關產品。包括提供內容意見回饋的位置、程式碼連結，以及您可能會覺得實用的各種文件。

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 12 堂 (START) 和第 13 堂 (FINISH) 模組的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
模組 12	code	code
單元 13 (本程式碼研究室)	code	code

線上參考資料

以下是可能與本教學課程相關的線上資源：

App Engine

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- App Engine [定價](#)和[配額](#)資訊

App Engine NDB 和 Cloud NDB

- [App Engine NDB 總覽](#)
- [App Engine NDB Datastore 用量](#)
- [Google Cloud NDB 說明文件](#)
- [Google Cloud NDB 存放區](#)
- [Cloud Datastore 定價資訊](#)

App Engine Memcache 和 Cloud Memorystore

- [App Engine Memcache 總覽](#)
- [Python 2 App Engine `memcache` 參考資料](#)
- [Python 3 App Engine `memcache` 參考資料](#)
- [App Engine `memcache` 至 Cloud Memorystore 遷移指南](#)
- [Cloud Memorystore 說明文件](#)
- [Cloud Memorystore for Redis 說明文件](#)
- [Cloud Memorystore for Redis 定價資訊](#)
- [Cloud Memorystore 支援的 Redis 版本](#)
- [Cloud Memorystore 首頁](#)
- [在 Cloud 控制台中建立新的 Memorystore 執行個體](#)
- [Python Redis 用戶端首頁](#)
- [Python Redis 用戶端程式庫說明文件](#)

Cloud VPC

- [Google Cloud VPC 說明文件](#)
- [Google Cloud 虛擬私有雲首頁](#)
- [Cloud VPC 定價資訊](#)
- [在 Cloud 控制台中建立新的無伺服器 VPC 存取連接器](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK \(`gcloud` 指令列工具\)](#)
- [所有 Google Cloud 說明文件](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
